

AirLynk

AI-native airline ecosystem backend, built as an independent exploration of AI-assisted passenger workflows, transactional airline systems, and backend engineering.

AirLynk — AI Airline Ecosystem Backend	Production-shaped portfolio / experimental backend	Agentic AI Engineer / Python Backend Engineer
Core stack	Purpose	Repository
Python, FastAPI, SQLAlchemy, PostgreSQL/SQLite, Gemini, JWT, Stripe, APScheduler	Explore how AI can become part of an airline workflow while keeping business-critical data deterministic and backend-controlled.	abizaid24/AirLynk-AI-Ecosystem-Backend

A. Full Case Study

01 — Overview

Project AirLynk — AI-Powered Airline Backend Ecosystem

AirLynk is an AI-based airline backend designed to explore how artificial intelligence can be meaningfully integrated into the airline and passenger experience. The project goes beyond a conventional booking API by connecting a Gemini-powered assistant to real airline functions while maintaining a deterministic transactional core.

The system covers flight discovery, fare classes, passenger and order workflows, seat maps and locking, payments, loyalty, pricing, notifications, scheduled automation, and an AI assistant that can retrieve verified airline information through approved function calls.

The central question behind the project was simple:

What would an airline experience look like if AI were treated as a core part of the system rather than just an additional feature?

02 — The Problem

Traditional airline journeys are built around structured pages and sequential steps: search, select, enter passenger details, choose seats, pay, and manage the booking. These workflows are functional, but they also create a large interaction surface for passengers to navigate.

The opportunity explored by AirLynk was not simply “add a chatbot.” It was to investigate whether AI could understand travel intent, retrieve real airline information, explain choices, and assist around the workflow without becoming the source of truth for prices, availability, or booking state.

03 — Research & Discovery

The project began with research into international airline digital experiences, including airline websites, flight search, booking, passenger information, seat selection, services, customer assistance, and the role of AI.

The research was used to understand the passenger journey and identify places where an AI-first interaction could potentially reduce friction or make the experience more contextual. The research framing in the original project notes deliberately focused on challenging the existing workflow rather than copying an existing airline interface.

Traditional journey

Search → Select Flight → Passenger Details → Seat Selection → Payment → Manage Booking

The design question

Instead of treating booking as a fixed sequence of forms, could an intelligent system participate around that process while the backend continues to enforce the actual airline rules and state?

04 — Product Vision

AirLynk's premise is closer to **AI-Native Airline Experience** than **Traditional Airline System + AI Chatbot**. Gemini is used as an interaction and orchestration layer, while the application services and database remain authoritative for airline facts and transactions.

The implemented AI assistant is designed to help passengers:

- search real flights using route/date information;
- understand fare and baggage rules;
- retrieve real order / booking status;
- receive budget-aware recommendations from current inventory;
- create approved price alerts;

The assistant is explicitly prevented from claiming that it completed payment or booking confirmation. Those remain explicit application actions.

05 — Solution

The implemented flow can be summarized as:

**User message → Gemini + approved tools → backend function → real database result
→ Gemini response**

For transactional workflows, the flow remains deterministic:

**Flight search → Order → Passenger / fare validation → Seat lock → Payment →
Booking confirmation**

06 — Technical Architecture

The backend is organized around FastAPI routers, service classes, SQLAlchemy models, authentication/security utilities, and scheduled background jobs. The AI service is a separate domain that calls existing airline capabilities rather than reimplementing the business logic inside prompts.

FastAPI routers	HTTP endpoints, request/response schemas and dependency injection.
Services	Business logic for AI, flights, orders, seats, pricing, payments, authentication, email and schedules.
SQLAlchemy models	Users, flights, airports, fare classes, seats, orders, bookings, passengers, payments, loyalty, alerts and AI conversations.
Database	PostgreSQL-compatible relational persistence; SQLite is supported by the default configuration.
Scheduler	Expired seat-lock cleanup, price-alert checks and rolling flight schedule maintenance.
External services	Gemini for AI, Stripe for payments, and email delivery for notifications.

Architectural principle

The system separates **AI interpretation** from **transactional authority**. Gemini can select an approved function, but the function executes application code against the actual database.

07 — AI Architecture

search_flights	Searches real AirLynk flight inventory by route/date/class.
get_order_status	Retrieves order / booking status by UUID or order reference.
explain_fare_rules	Explains change, refund and baggage rules for a fare class.
check_baggage_policy	Returns checked and cabin baggage allowance.
recommend_within_budget	Finds real bookable options within or near a user's budget.
set_price_alert	Creates an authenticated route / target-price alert.

08 — AI Reliability & Guardrails

The distinctive AI engineering choice is not merely that Gemini is connected to the backend; it is that the assistant is deliberately constrained from fabricating airline facts.

The AI system prompt states that flight data, prices, seat availability and booking status must come from the provided functions. Payment and booking confirmation must always be completed explicitly in the application.

Function-call execution is therefore the bridge between conversational intent and authoritative application state.

Additional reliability behavior implemented in the AI service:

- Gemini requests are time-limited and the service limits function-call rounds.
- If Gemini fails or times out, a rule-based fallback assistant can still use real backend functions.
- Conversation messages and executed function calls are persisted in AIConversation.
- IATA parsing is constrained against a whitelist of real seeded airport codes to avoid false route matches.
- Budget-related requests are directed toward the dedicated recommendation function rather than invented “cheap” options.

09 — Booking & Order System

AirLynk uses a unified order model for the transaction. An order can contain bookings, passengers, ancillary services and payments, while each booking represents flight-level details and a generated PNR.

Flight discovery	Search and filtering against actual flight / fare-class data.
Passenger handling	Passenger details are associated with the booking / order.
Ancillaries	Additional services contribute to order totals.
Pricing	Fare subtotal + ancillaries + taxes + transparent booking fees – loyalty discount.
Payment	Stripe PaymentIntent and webhook flow, plus a demo confirmation path.
Confirmation	Payment confirmation moves the booking into its confirmed state and finalizes selected seats.

10 — Seat Selection Challenge

Seat selection is implemented in the core backend independently of the AI assistant. The project uses database-backed locks to protect inventory during the period between selection and payment.

11 — Seat Locking & Concurrency

The seat-lock service uses row-level database locking with `SELECT . . . FOR UPDATE`. A seat can be temporarily locked with a user ID and expiration timestamp, then converted to `BOOKED` after payment.

AVAILABLE	Eligible for a temporary lock.
LOCKED	Stores <code>locked_until</code> and <code>locked_by_user_id</code> ; another user cannot take the lock.
BOOKED	Permanently unavailable through the normal locking path.
BLOCKED	Unavailable for booking.

Lock lifecycle

Acquire row lock → validate state → expire stale lock if needed → set `locked_until` → payment → `BOOKED`

Expired locks are also released by a scheduled cleanup job. Release operations verify ownership unless a forced administrative flow is used.

12 — Pricing, Loyalty & Carbon

The `PricingService` centralizes the arithmetic needed by the booking domain rather than spreading pricing logic across routes.

Fare classes	Economy, Premium Economy, Business and First multipliers.
Taxes	12% tax applied transparently to subtotal.
Booking fee	\$5 per passenger.
Loyalty redemption	100 points = \$1 discount, capped against payable amount.
Points earned	Class multiplier increases loyalty points for higher fare classes.
Dynamic pricing	Demo demand adjustment based on occupancy and days to departure.
Carbon	Distance × fare-class carbon rate.
Price history	Current and historical pricing can be stored for flight classes.

13 — Automation

`APScheduler` runs operational jobs such as expired seat-lock cleanup, active price-alert checks, and rolling flight-schedule maintenance. This allows time-based system state to be maintained without requiring a user request.

14 — Backend Engineering

The repository treats AirLynk as a real backend system rather than a collection of demo endpoints. Authentication, validation, service boundaries, persistence, payments, scheduled work and exception handling are all represented as separate concerns.

Authentication	JWT access/refresh token flow with protected user/admin operations.
Validation	Pydantic schemas and service-level business validation.
Errors	Dedicated exception types and centralized handling.
Persistence	Async SQLAlchemy sessions and relational models.
Migrations	Alembic-based schema migration setup.
Infrastructure	Dockerfile and docker-compose configuration.
API documentation	FastAPI OpenAPI / Swagger and ReDoc.
Testing	pytest / pytest-asyncio test coverage across key domains including AI, flights, orders and auth.

15 — Key Engineering Challenges & Decisions

- 1. Making AI useful without making it authoritative.** The assistant needed real airline data but should not become a free-text source of truth. Function calling was used as the controlled bridge.
- 2. Keeping seat inventory safe under concurrency.** Temporary locks need ownership, expiration and database-level protection. Row-level locking was used instead of relying on a purely in-memory mechanism.
- 3. Connecting payment state to booking state.** Payment confirmation must result in a deterministic booking transition and permanent seat confirmation.
- 4. Keeping pricing logic centralized.** Taxes, fees, loyalty discounts, dynamic adjustments and carbon estimates belong in a service rather than being duplicated across endpoints.
- 5. Supporting AI failure.** Gemini timeouts and failures are handled explicitly, with a fallback path for supported informational queries.

The deeper lesson

The hard part was not adding an LLM. The hard part was making an AI interaction layer coexist with structured, stateful and transactional airline workflows.

16 — What I Learned

AirLynk reinforced several engineering lessons that shaped the final architecture:

- AI works best as an orchestration and interpretation layer when deterministic business logic remains the source of truth.
- Tool/function calling is more useful than free-form generation when the assistant needs access to structured business data.
- Concurrency problems such as seat allocation require database-level guarantees, not only application-level checks.
- Payment, inventory and booking confirmation should remain deterministic even when the surrounding user experience becomes conversational.
- A production-shaped AI system needs failure paths, validation, persistence, logging and testing in addition to prompts.

17 — Implemented vs. Conceptual

AI flight search	Implemented	Gemini function calls real flight-search logic.
AI order status	Implemented	Assistant can retrieve real order / booking state.
Fare & baggage explanation	Implemented	Dedicated approved functions expose policy information.
Budget recommendations	Implemented	Recommendations come from current backend inventory.
Price alerts	Implemented	AI can create approved authenticated alerts.
Seat locking	Implemented	Core backend uses database row locks and expiry.
Stripe payments	Implemented	PaymentIntent + webhook + demo confirmation path.
AI autonomous payment	Conceptual / not implemented	AI is explicitly prevented from claiming payment completion.
AI end-to-end autonomous booking	Not established	Booking exists in backend, but full booking completion is not exposed as an AI function.
AI-controlled seat selection	Not established	Seat selection exists in backend; direct AI seat-selection tooling is not exposed.

18 — Project Impact

Qualitatively, AirLynk demonstrates an independently built backend that combines conversational AI, structured airline services, transactional inventory management, payments, pricing logic and scheduled automation. No performance or business metrics are claimed here because the repository does not establish production-scale measurements.

19 — Future Improvements

The following are logical future directions rather than claims of completed functionality:

- Expose additional carefully controlled airline actions to the AI assistant, such as guided booking steps, while retaining explicit confirmation for payment and irreversible actions.
- Add richer multi-turn booking state so the assistant can help assemble a trip before handing control back to the application for confirmation.
- Expand automated integration and concurrency tests around seat locking, payment webhooks and complete booking flows.
- Introduce production-grade observability and deployment infrastructure beyond the current Docker-based setup.
- Expand airline-data integrations beyond the seeded / application-managed inventory model.

20 — Technology Stack

Backend	Python, FastAPI, Uvicorn
Database	SQLAlchemy Async, PostgreSQL / SQLite
AI	Google Gemini, function calling
Auth & security	JWT, password hashing, dependency-based authorization
Payments	Stripe PaymentIntents + webhooks
Scheduling	APScheduler
Migrations	Alembic
Testing	pytest, pytest-asyncio, HTTPX
Infrastructure	Docker, docker-compose

21 — My Role

Built as an independent backend / AI engineering project. The work covered the product-to-engineering loop: researching airline experiences, framing the AI-native concept, designing the backend architecture, implementing AI function calling, building airline workflows, handling seat concurrency, integrating payments, implementing pricing and automation, and testing the resulting system.

The project demonstrates a combination of **Product Thinking + AI Engineering + Backend Engineering + System Design + Problem Solving**.

22 — Project Summary

AirLynk began with an exploration of how airline digital experiences could change if AI were treated as a core system component. The resulting backend connects a Gemini-powered travel assistant to real airline functions while keeping flight inventory, booking state, seat availability, payments and pricing under deterministic application control.

The central engineering idea is simple: let AI interpret and orchestrate; let the backend remain authoritative.

B. Portfolio Version — AirLynk

AirLynk is an AI-powered airline backend ecosystem built to explore AI-native passenger experiences. Gemini function calling connects conversational requests to real flight search, order-status, fare-rule, baggage, budget-recommendation and price-alert capabilities, while a FastAPI + SQLAlchemy backend handles booking workflows, database-backed seat locking, pricing, loyalty, payments, authentication and scheduled automation.

Stack: Python · FastAPI · SQLAlchemy · PostgreSQL/SQLite · Google Gemini · JWT · Stripe · APScheduler · Docker

C. Project Summary / Card

AirLynk explores what an airline experience could look like when AI is integrated into the system rather than bolted on as a chatbot. The backend combines Gemini function calling with real airline services, transactional booking logic, seat locking, payments, dynamic pricing, loyalty and automation — with the database and application services remaining the source of truth.

D. Key Highlights

- Gemini function-calling travel assistant grounded in real backend data
- Flight search, order-status, fare-rule and baggage tools
- Budget-aware recommendations and authenticated price alerts
- Database row-level seat locking with expiry and ownership
- Stripe payment flow with deterministic booking confirmation
- Dynamic pricing, loyalty points and carbon estimation
- APScheduler automation for locks, alerts and rolling schedules
- FastAPI + SQLAlchemy backend with authentication, migrations, testing and Docker

E. Case Study Metadata

Project title	AirLynk
Tagline	An AI-native airline backend that connects conversational intelligence to real airline workflows.
Role	Agentic AI Engineer / Python Backend Engineer
Category	AI-powered airline ecosystem / backend engineering
Repository	github.com/abizaid24/AirLynk-AI-Ecosystem-Backend